

FastSDPCertification

Experiment Summary — McCormick “composed” vs baseline on cross-product terms

Network blob_nn_4x10-1 — 330 cross-product terms tested individually

1 Objective

In the SDP relaxation used to certify the `blob_nn_4x10-1` network, every cross-product term of two neurons $z_{l,u} \cdot z_{k,j}$ can be relaxed in two ways: the baseline strategy (`one_variable`, also called `none/simple` in the files), or a finer but more expensive McCormick relaxation, `composed`. The experiment aims to determine, term by term, whether switching to `composed` actually improves the certification rate and the optimal value of the SDP program, and whether this choice can be predicted automatically from the bounds (LB/UB) of the two neurons involved, without having to test every single term with the solver. This report walks through the analysis in the order it was actually done: first at the level of per-combo means, then at the level of individual data points, then with a trained decision tree — and ends on why, in practice, `one_variable` everywhere turns out to be the most defensible default.

2 Methodology

- **Baseline** (`combo_0000`): all 330 cross-product terms of the network are set to `one_variable`.
- **330 test combos**: each `combo_XXXX` starts from the baseline and flips exactly one term (layer l , neuron `u_idx`) $\times$ (layer k , neuron `j_idx`) to `composed`.
- Each combo is evaluated on a test set of 78 to 80 data points; for every point we record `certified` (`optimal_value > 0`), `optimal_value`, and the `gain` relative to the baseline on that same point. **Definition:** `gain = optimal_valuecombo - optimal_valuebaseline`, computed point by point (same `data_index`, same combo’s own bounds vs. the baseline’s optimal value on that exact point) — so `gain` isolates the effect of switching this one term to `composed`, independently of how easy or hard that particular test point is to certify in the first place.
- **Computational cost.** Every one of these solves goes through MOSEK, and MOSEK is genuinely slow at this scale: the project’s cumulative `Mosek_logger.log` has grown to **2.8 GB** and records **14,608 individual solver calls** spread over roughly **two months** of intermittent runs (9 June – 11 August 2026) across the project’s various experiments. Each single MOSEK call is quick on its own (typically 0.03–0.1s, per the log), but with 330 candidate terms $\times$ 78–80 test points, a single full pass of this experiment requires tens of thousands of separate solves — that per-call overhead and queueing, not the solver’s raw speed, is what makes a full combo sweep take hours to run in practice. This is precisely the motivation for trying to replace solver calls with a lightweight predictor: it would remove the need to re-run MOSEK for every candidate term.

3 First pass: working with the mean of each combo

The first analysis pass collapsed every combo down to a single row: the mean `optimal_value/gain` over its test points, and the mean LB/UB bounds of its two neurons

over that same test set. This section presents that pass in full, then explains why its results were not very conclusive.

3.1 Aggregate comparison: composed vs baseline

Across the 130 successfully tested combos (with 3 baseline runs kept as reference), **composed** brings, on average, no net benefit:

Strategy	Mean certification rate	Mean optimal value
Baseline (none)	50.9%	−3.03
Composed (average of 130 combos)	49.9%	−3.18

3.2 Separating combos by their mean optimal value (separation_plot.ipynb)

`separation_plot.ipynb` looks inside the 130 composed rows: it trains a depth-4 decision tree (`min_samples_leaf=3`) on `l`, `u_idx`, `k`, `j_idx`, `LB_neuron1`, `UB_neuron1`, `LB_neuron2`, `UB_neuron2` (all of them **means over the test set**) to predict `optimal_value_avg`, ranks the features by importance, and plots the tree’s predicted surface over its top-2 features with a “composed cutoff” line at mean $+1\sigma$ of the predicted values.

Re-running this exact pipeline gives `UB_neuron2` (importance 0.605) and `u_idx` (0.388) as by far the two most important features — `l`, `k`, `j_idx` score 0.000. Over the 130 composed combos, `optimal_value_avg` ranges from -3.203 to -2.731 (mean -3.179 , std 0.056), and the cutoff line sits at -3.150 .

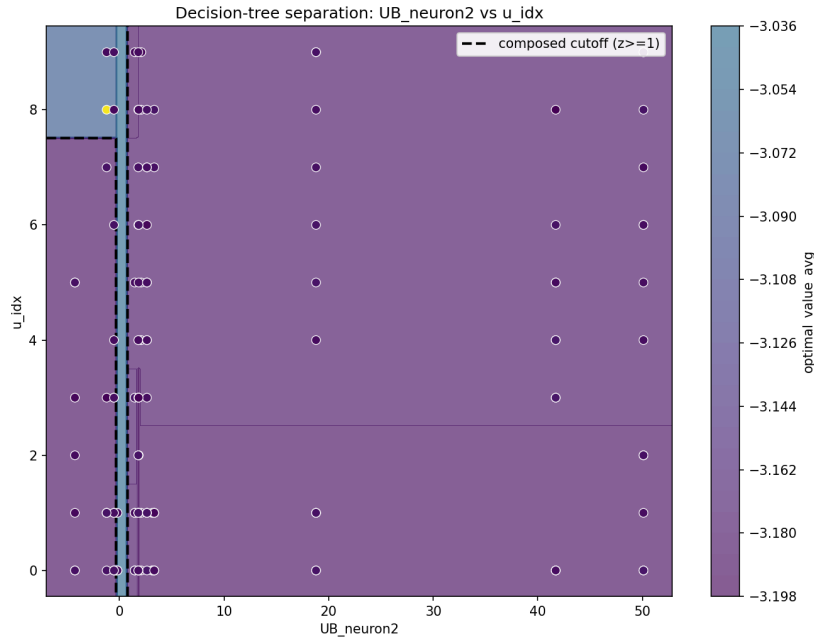


Figure 1: `separation_plot.ipynb` output: `UB_neuron2` vs. `u_idx` (mean bounds), coloured by `optimal_value_avg`. The background is almost uniformly the same shade (≈ -3.15 to -3.20) except for a thin vertical band near `UB_neuron2` ≈ 0 .

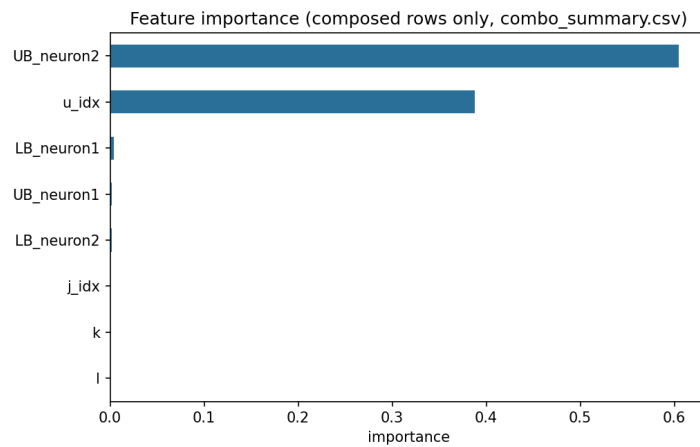


Figure 2: Feature importance of that tree (130 composed rows): `UB_neuron2` and `u_idx` carry essentially all of it; `l`, `k`, `j_idx` contribute exactly 0.

3.3 Distribution of the mean gain per combo

The mean gain (composed $-$ baseline) per combo is almost always close to zero, and slightly negative for the majority of combos: out of 158 aggregated combos, 92 have a negative or zero mean gain.

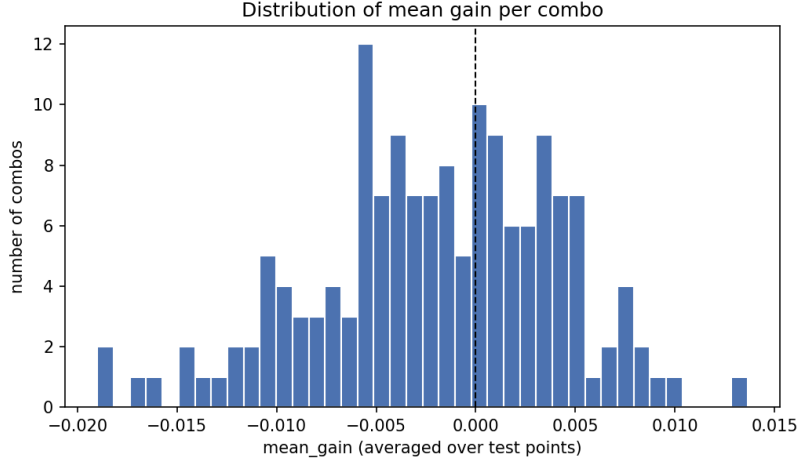


Figure 3: Distribution of the mean gain per combo.

3.4 Applying the (later) tree rules to per-combo means

As a sanity check, the fixed rules of the trained gain-regression tree from Section 5 were applied to these same per-combo **mean** bounds instead of per-point bounds (`classify_combos_from_runs.py`, output `combo_classification.csv`). Result: **309 out of 309** evaluated combos get a predicted gain of exactly 0.00 and are labelled **simple** (`one_variable`) — not one single combo’s mean bounds fall inside the narrow window that the tree associates with a positive gain.

3.5 Interpretation: not very conclusive, and the bounds are not rigorously defined

Two things stand out from this first pass:

- **The results are not very conclusive.** The separation found (Fig. 1) is real but narrow, the aggregate composed/baseline comparison shows no net benefit, and applying the eventual tree rule to these same per-combo figures recommends **one_variable** for literally every combo (Section 3.4). Nothing here points decisively towards **composed** being worth it anywhere.
- **“The bounds are the most important feature” is not a rigorous statement at this level.** `UB_neuron2` tops the feature-importance ranking (Fig. 2), but what is actually being ranked is the **mean** of `UB_neuron2` over 78–80 test points — a quantity the solver never sees directly (it only ever sees one point’s exact bound at a time). Why average at all, rather than, say, using the tightest or widest bound actually reached across the test set? Nothing in this pipeline justifies the mean specifically over any other summary statistic, so “the bounds matter” is true only in a loose sense here: it tells us bounds are *worth looking at*, not that this particular (mean) definition of them is the right one to act on.

This is what motivated moving to the per-data-point analysis below, where “the bounds” finally have an unambiguous meaning: the exact LB/UB of the two neurons, for that one specific test point.

4 Second pass: using every single data point

This reproduces the pipeline of the `combo_visualisation.ipynb` notebook (Section 7), run directly on `combo_detail_by_datapoint.csv` (18,018 usable rows after cleaning, one row per test point per combo — no averaging, so `LB_neuron1`, `UB_neuron1`, `LB_neuron2`, `UB_neuron2` are now each point's own exact bounds). A depth-3 regression tree is trained, with a 75/25 train/test split (`random_state=0`), to predict the continuous `gain`.

4.1 Gain regression

The tree trained on `LB_neuron1`, `UB_neuron1`, `LB_neuron2`, `UB_neuron2` achieves an R^2 of **0.027 on the test set** (0.029 on train): it does capture a real spread between leaves (-0.09 to $+0.23$), but only explains $\approx 3\%$ of the variance in `gain` — most of the point-by-point noise is left unexplained. Adding `k`, 1 changes nothing (test $R^2 = 0.026$).

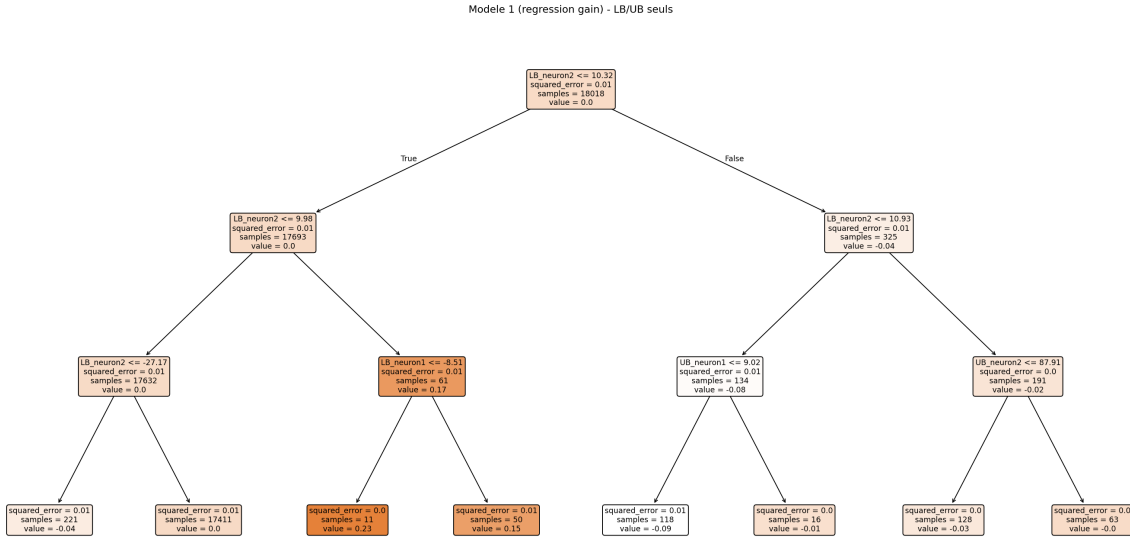


Figure 4: Regression tree (gain), depth 3, on the per-point `LB_neuron1`, `UB_neuron1`, `LB_neuron2`, `UB_neuron2`. Test $R^2 = 0.027$.

4.2 Interpretation

Using every single data point instead of per-combo means resolves the ambiguity from Section 3: the bounds here are exact, not averaged, so a feature-importance ranking on them is a statement about actual solver inputs. Even so, the gain model (Fig. 4), which is the one that actually decides `composed` vs `one_variable`, stays weak ($R^2 \approx 0.03$) — moving from combo-level means to exact, per-point bounds removed the definitional ambiguity from Section 3, but it did not turn this into a strong predictor of gain.

5 The trained tree, and where the gain drops away

The gain-regression tree from Section 4.1 is the model actually used to decide `composed` vs `one_variable` per term. Its rules:

Rule (learned thresholds, gain model)	Samples	Predicted gain
$\text{LB_neuron2} \leq -27.17$	221	-0.040
$-27.17 < \text{LB_neuron2} \leq 9.98$	17,411	+0.004
$9.98 < \text{LB_neuron2} \leq 10.32$ and $\text{LB_neuron1} \leq -8.51$	11	+0.229
$9.98 < \text{LB_neuron2} \leq 10.32$ and $\text{LB_neuron1} > -8.51$	50	+0.161
$10.32 < \text{LB_neuron2} \leq 10.93$ and $\text{UB_neuron1} \leq 9.02$	118	-0.089
$10.32 < \text{LB_neuron2} \leq 10.93$ and $\text{UB_neuron1} > 9.02$	16	-0.011
$\text{LB_neuron2} > 10.93$ and $\text{UB_neuron2} \leq 87.91$	128	-0.032
$\text{LB_neuron2} > 10.93$ and $\text{UB_neuron2} > 87.91$	63	-0.001

Only two of these eight leaves are positive, and they both require `LB_neuron2` to sit inside a window barely 0.34 wide (9.98 to 10.32); every other leaf — covering the rest of the real line — is at or below zero. The `samples` column makes the imbalance concrete: the two positive leaves together hold only $11 + 50 = 61$ of the 18,018 training points (0.34%), while the single largest leaf ($-27.17 < \text{LB_neuron2} \leq 9.98$, 17,411 points, 96.6% of the data) predicts an essentially-zero gain of +0.004. This is the “drop in gain” visible at every level of this report:

- at the raw per-point level, the best a single leaf ever predicts is +0.229 (on just 11 points), next to leaves as low as -0.089 (on 118 points);
- once bounds are averaged per combo instead of taken per point (Section 3.4), the gain drops to exactly 0.00 for **all 309** combos checked — the narrow positive window essentially never survives averaging, because it is 0.34 wide against a range of tens of units.

The specific, narrow condition under which `composed` is predicted to actually help essentially never holds once you can no longer cherry-pick a single favourable data point in advance.

6 Validating the tree on a real run (`compare_runs.ipynb`)

Everything up to this point was cross-validation on historical solver data. `compare_runs.ipynb` instead runs the actual pipeline with `relu_relaxation_type: "auto"` — meaning the trained gain-tree from Section 5 decides `composed` vs `one_variable` live, term by term, from each point’s own bounds — and compares its solver output directly against a `one_variable`-everywhere baseline, point by point.

6.1 Setup

- **Comparison run:** `relu_relaxation_type: "auto"`, executed 2026-08-11 on `blob_nn_4x10-1` over a 1000-image batch (872 points actually recorded, in five parts).
- **Baseline:** two available `combo_0000` (all-`one_variable`) runs, both from 2026-08-07, each with 80 points (`data_index` 0–79).
- **Matching:** following `compare_runs.ipynb` exactly, both sides are sorted by `data_index` and truncated to the first 78 points (the same test-set size used everywhere else in this report). All 78 `data_index` values coincide exactly between baseline and comparison (same 78 images), so this is a genuine point-by-point comparison, not a positional guess.

6.2 Results

Using the first baseline run as the reference:

Quantity	Value
Matched points	78
Mean gain (auto – baseline)	−0.178
Median gain	−0.116
Points with gain > 0	4 (5.1%)
Points with gain < 0	73 (94.9%)
Largest positive gain	$+3.7 \times 10^{-7}$
Most negative gain	−0.804
Newly certified (baseline no → auto yes)	0
Newly lost (baseline yes → auto no)	4
Certification rate, baseline	52.6%
Certification rate, auto (tree)	47.4%

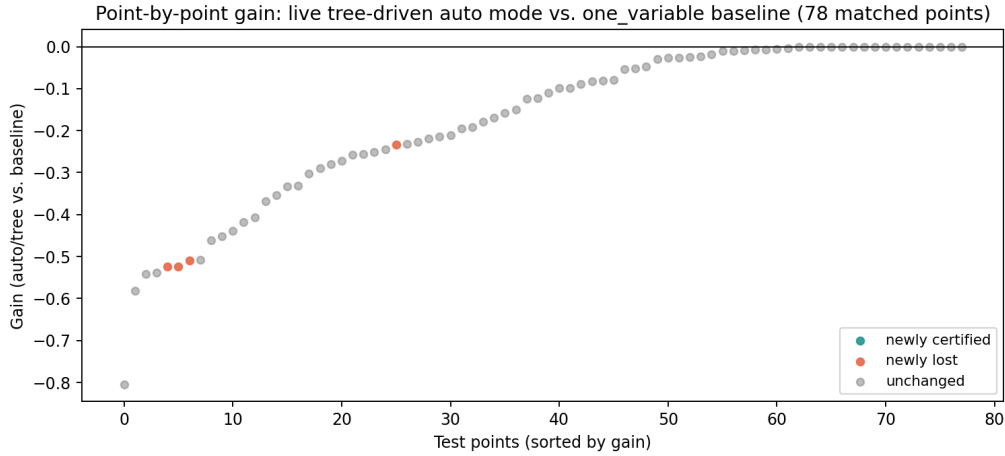


Figure 5: Point-by-point gain of the live, tree-driven *auto* mode relative to the *one_variable* baseline, on the 78 matched test points. Almost every point is at or below zero; the few points shown above zero are of order 10^{-7} – 10^{-8} (solver noise, not a real gain).

The four points with a nominally positive gain are all of order 10^{-7} to 10^{-8} — far below anything that could be called a real improvement. Re-running the same comparison against the *second* available baseline run gives the same picture (mean gain -0.179 , certification rate $52.6\% \rightarrow 47.4\%$, the same 4 points newly lost and 0 newly gained); the one point that looked nominally positive against that second baseline ($+0.031$) is smaller than the ≈ 0.33 run-to-run variation MOSEK itself produces between the two baseline runs on that exact same image, so it cannot be distinguished from ordinary solver noise either.

6.3 Interpretation

This is the one part of the analysis that is not cross-validation or leaf statistics: it is the trained tree actually driving solver decisions on a real, previously-unseen batch of 872 images, compared point by point against always using *one_variable*. The result matches every earlier warning sign in this report — the mean gain is clearly negative, the certification rate drops by 5.2 points, and not a single point shows a gain large enough to distinguish from solver noise. Letting the tree decide live does not merely fail to help; on this run it makes both the optimal value and the certification rate worse than the *one_variable* baseline.

7 Conclusions

- **First pass (per-combo means, Section 3):** not conclusive — `composed` shows no net benefit on average, and the “important” bounds identified there are means over the test set, a quantity with no clear operational meaning (the solver never sees a mean bound). This motivated moving to per-point data.
- **Second pass (per data point, Section 4):** with bounds unambiguously defined, exactly rather than averaged, the `gain` model — the one that actually drives the `composed` decision — still stays weak ($R^2 \approx 0.03$).
- **The trained tree (Section 5):** only a 0.34-wide window of `LB_neuron2` predicts a positive gain; everywhere else is at or below zero, and that window does not survive averaging bounds per combo (0 of 309 combos recommended `composed` when checked that way).
- **Validating the tree on a real run (Section 6):** letting the tree decide `composed` vs `one_variable` live, point by point, on a real 78-point held-out batch makes things worse, not better — mean gain -0.178 , certification rate down from 52.6% to 47.4%, and not one point shows a gain that rises above solver noise. This is not an extrapolation from cross-validated statistics: it is the tree’s own live decisions performing worse than the fixed `one_variable` baseline.
- **MOSEK is already the practical bottleneck** (Section 2): 14,608 individual solves and 2.8 GB of logs accumulated on this project. Chasing a narrow, unreliable gain window is not worth the additional solver calls it would cost to validate per term.
- **Conclusion:** given that (a) `composed` shows no benefit on average, (b) its gain model is weak and its one favourable region is a sliver that disappears under any reasonable aggregation, and (c) letting the tree drive decisions live on a real run actually reduced the certification rate, using `one_variable` for every single cross-product term is the most reasonable, defensible default — at least until a dedicated experiment demonstrates a targeted `composed` choice that actually holds up.
- **Limitation:** the tree and its thresholds were learned on a single network (`blob_nn_4x10-1`) and a single test set; their generalization to other architectures or datasets still needs to be checked. In practice, this is largely a consequence of MOSEK’s own runtime: at 14,608 solves and 2.8 GB of logs already spent on this one network (Section 2), repeating the same combo-by-combo sweep on larger or additional networks was not feasible within the time available for this experiment.

Sources: `scripts/combo_summary.csv`, `combo_summary_aggregated.csv`,
`combo_classification.csv`, `combo_detail_by_datapoint.csv`, `separation_plot.ipynb`,
`classify_combos_from_runs.py`, `bound_type_predictor.py`, `compare_runs.ipynb`,
`results/benchmark/blob_nn_4x10-1/2026_08_11_00h46_41s/` (auto/tree run),
`results/benchmark/blob_nn_4x10-1/loop_705_combo_0000__baseline__all_one_variable/`
and `loop_873_...` (baseline runs), `results/Mosek_logger.log` — `scripts/` and `results/` folders of
the *FastSDPCertification* project.